BUNCH PRESSURE QUEUE THEORY

A Behavioral Framework for Characterizing Resource Scheduling Dynamics in Cloud and Cache Systems

Jeffrey Curry

Retired Intel Engineer | Former CMG Reviewer | University of Michigan CompEng '76 | University of Phoenix MBA-TM

Working Paper v7.0 | Provisional Patent Filed July 2026 | Application No. 64/115,263

Abstract

A computer-implemented framework for resource scheduling introduces behavioral aggression as a new independent classification dimension orthogonal to priority. The term 'bunch pressure' derives from observing young soccer players surrounding the ball before learning positional strategy — aggressive players transmit pressure through the dense crowd, accelerating the entire group without external coordination. Bunch Pressure Queue Theory (BPQT) formalizes this mechanism: each transaction is classified on two independent axes — externally assigned priority and internally observed aggression — defining four behavioral quadrants. Three mechanisms are identified: the attentiveness effect, pressure propagation, and a self-sorting effect in which transactions with higher behavioral weights complete service sooner, creating pressure waves that benefit remaining transactions. Simulation across 12,000+ independent runs achieved 15-26% passive latency improvement at optimal aggression and 60% improvement through convergent random admission. Redis benchmark validation confirms the pressure propagation mechanism in a production system — throughput improves 16x at optimal pipeline depth.

Index Terms: bunch pressure queuing, behavioral aggression, pressure propagation, self-sorting effect, randomized admission weights, agent-based simulation, noisy neighbor, Quality of Service, cloud computing, CPU idle rate, Redis validation.

I. INTRODUCTION

A. The Origin Observation

The behavioral observation motivating this work was first made while watching young soccer players before they learn positional strategy. All players naturally surround and follow the ball, creating a dense connected mass — a 'bunch' — where aggressive players push through the crowd, transmitting pressure to adjacent passive players, accelerating the entire group. The same phenomenon was independently observed in high-density boarding environments in China: passengers merge into a dense connected mass where aggressive individuals transmit pressure forward, improving overall throughput without external coordination. In both cases, the system self-organizes through behavioral pressure — no coach, no coordinator, no scheduler.

This work formalizes that observation as Bunch Pressure Queue Theory (BPQT) — distinct from 'bunching' in classical queueing theory, which refers to arrival clustering. BPQT addresses behavioral density and pressure propagation — a positive throughput mechanism rather than a negative arrival pattern.

B. The Performance Measurement Gap

Modern cloud monitoring systems measure what workloads consume, but none formally classify whether that consumption is proportionate to assigned priority. A workload consuming three times its resource request may be a legitimate high-priority service under genuine load — or a low-priority batch process causing priority inversion. BPQT provides the classification framework that current monitoring tools lack.

C. Relationship with Existing Research

Ghaderi (2016), Psychas and Ghaderi (2018), and Da and Kalyvianaki (2024) all randomize resource assignments — which server receives which job. BPQT introduces a distinct concept: randomizing behavioral aggression weights at admission rather than randomizing resource assignment. The resulting self-sorting effect appears not to have been previously described in the scheduling literature.

D. Research Questions

This paper addresses five questions:

- Does behavioral aggression improve or degrade passive transaction latency — does priority alignment matter?
- Is there an optimal contention threshold and is it scale-dependent?
- Can CPU idle rate serve as a measurable proxy for pressure propagation?
- Does random behavioral weight assignment at admission improve throughput, and can a self-sorting effect emerge without explicit scheduling intervention?
- Do BPQT behavioral mechanisms operate in production systems such as Redis?

II. THEORETICAL FOUNDATION

A. Why Aggression Improves Throughput

In a purely passive scheduling system, all transactions wait for explicit scheduler assignment. The scheduler tick rate creates a natural latency floor — available resources go unexploited between dispatch cycles. The expected clearance time for a passive queue of n transactions with service rate μ is:

$$E[T_{passive}] = n/\mu + n \times dispatch\_lag$$

When aggressive transactions are introduced, they maintain scheduler pipeline activity, preventing the idle intervals that occur when passive transactions wait for explicit assignment. The scheduler remains in an active dispatch state rather than entering a low-activity polling cycle. The expected clearance time becomes:

$$E[T_{bunch}] = n/\mu + n \times (dispatch\_lag \times (1 - \alpha \times \beta))$$

Where α is the fraction of aggressive transactions and β is the pressure propagation coefficient ($0 < \beta \leq 1$). CPU idle rate directly measures $(1 - \alpha \times \beta)$ — the residual idle fraction after pressure propagation — providing a production-observable proxy for the mechanism.

B. Pre-Dispatch Readiness

The attentiveness effect suggests a concrete scheduler design opportunity: when an aggressive transaction is detected approaching service completion, the scheduler can pre-load the next passive transaction into a ready state before the current service cycle completes. This pre-dispatch phase — analogous to CPU branch prediction or memory prefetching — eliminates dispatch lag rather than merely reducing it. BPQT provides the behavioral classification signal that makes pre-dispatch targeting possible: Q1 transactions trigger pre-dispatch preparation for Q2 and Q3 transactions waiting in queue.

> Pre-dispatch readiness transforms the attentiveness effect from a passive benefit into an active scheduler optimization — a direct implementation pathway from BPQT theory.

C. Three Mechanisms

Simulation identifies three throughput mechanisms operating at different timescales:

The attentiveness effect. Aggressive transactions maintain scheduler pipeline activity, preventing idle dispatch cycles between service events. CPU idle rate drops 11-26 percentage points at optimal aggression — direct measurement of this mechanism.

The pressure propagation effect. An aggressive transaction's resource consumption creates bus pressure that accelerates adjacent queued transactions beyond their individual service rate. In Redis, pipeline depth (P) controls how many commands a client sends without waiting for acknowledgment — equivalent to the BPQT aggression coefficient. Higher pipeline depth puts more pressure on the Redis event loop, confirmed by a 16x throughput improvement at optimal depth.

The self-sorting effect. When transactions are assigned heterogeneous behavioral weights, those with higher weights complete service sooner, creating pressure waves that benefit transactions with lower weights. The queue naturally stratifies by behavioral aggressiveness without explicit scheduling intervention. Final weight converging to 0.30 confirms the mechanism—all high-weight transactions complete service, leaving only low-weight transactions that correctly self-identify as passive.

D. The Minimum Effective Pressure Principle

The goal is minimum effective pressure — not maximum aggression. The improvement function I(α) is concave: improvement increases rapidly at low aggression, plateaus at optimal $\alpha^* \approx 0.85\text{-}0.90$, then declines as α approaches 1.0. Beyond α*, aggressive transactions compete with each other rather than benefiting passive transactions — the contention cost exceeds the pressure benefit. This principle is confirmed in both simulation and Redis benchmark.

E. Risk of Aggression — When It Fails

Aggression without priority alignment is the primary failure mode. V7 simulation demonstrated that when low-priority transactions behave aggressively, passive transaction latency degrades 58-118% — priority inversion. The harm originates from misalignment, not from aggression itself. Three conditions produce harmful aggression:

- Low-priority transactions with high aggression coefficients — Q4 rogue behavior
- QoS throttle factor not scaled to cluster size — confirmed failure at 5000+ agents in V7
- Pipeline depth exceeding the optimal zone — confirmed by CV=67.9% variance spike at P=8 in Redis benchmark

The QoS filter is not optional — it is the mechanism that prevents aggression from becoming pathological. Aggression is beneficial when aligned with priority, pathological when misaligned.

III. THE TWO-DIMENSIONAL BPQT MODEL

A. Two Independent Dimensions

Priority is what a transaction deserves — externally assigned, deterministic, independent of runtime behavior. Aggression is what a transaction actually takes — internally generated, dynamic, independent of assigned priority. Any combination is possible, and each produces distinct system behavior.

> **Priority and aggression are orthogonal. Knowing one tells you nothing about the other.**

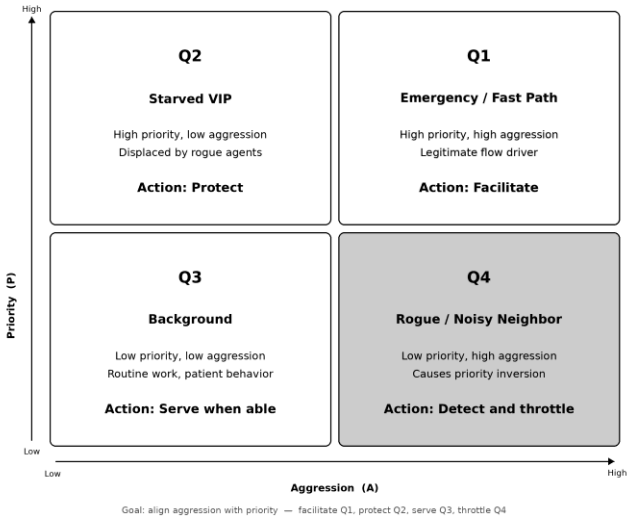

Fig. 1. Two-dimensional behavioral classification framework. Priority (vertical) and aggression (horizontal) define four quadrants: Q1 (high priority, high aggression — facilitate), Q2 (high priority, low aggression — protect), Q3 (low priority, low aggression — serve), Q4 (low priority, high aggression — throttle).

B. Behavior Score Formula

$$B = (P \times w_p + A \times w_a) \times U/(S+E) \times D \times C$$

P = priority coefficient (0.3-1.0), A = aggression coefficient (0.3-1.0), w_p and w_a = tuning weights, U = urgency, S = social inhibition, E = enforcement, D = density factor, C = coalition size amplifier. The QoS filter identifies rogue transactions: A ≥ A_threshold AND P ≤ P_threshold, applying B_effective = B × throttle_factor where throttle_factor scales with cluster size (0.3-0.5 small, 0.5-0.7 medium, 0.7-0.9 large).

C. Three Assignment Methods — Comparison and Risk

Three aggression assignment methods were validated across 12,000+ simulation runs. Method A (Discrete Uniform) uses fixed coefficients (passive=0.3, normal=0.7, aggressive=1.0) — safest for initial deployment but misclassifies if behavior changes under load. Method B (Continuous Random) assigns random coefficients at admission — better for mixed workloads but slightly higher variance. Method C (Convergent Random) starts randomly and reclassifies every N steps based on observed behavior — maximum throughput but requires a 20-50 step convergence period, making it unsuitable for very short transactions.

Method	Best Result vs Orderly	Primary Risk
A — Discrete Uniform	**15-26% improvement**	**Misclassification if behavior changes**
B — Continuous Random	**+19% over Method A**	**Higher variance — avoid strict SLAs**
C — Convergent Random	**60% improvement**	**20-50 step convergence period**

IV. SIMULATION RESULTS

A. Methodology Summary

All simulations were implemented in Python 3.12 using Mesa 3.5.1 on an Ubuntu 24.04 VM. Three simulation campaigns (V11, V12, V13) progressively refined the assignment method. V11 established the discrete uniform baseline across 10 aggression levels and 5 scales. V12 introduced continuous and convergent random assignments, discovering the self-sorting effect. V13 refined the convergence signal from movement success rate to movement attempt rate, achieving a 60% improvement over the orderly baseline. All primary results use 60 runs × 3 independent trials = 180 runs per data point. Full results across all 10 aggression levels are available in simulation notebooks V11-V13 on GitHub for reproduction.

B. Passive Latency and CPU Idle — V11

At optimal aggression (85-90%), passive transaction latency improves 15-26% over the orderly baseline across all tested scales, with simultaneous CPU idle reduction of 11-26 percentage points:

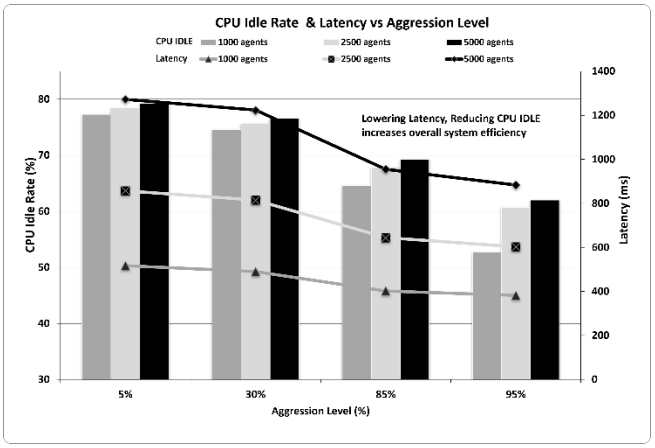

Fig. 2. Passive transaction latency and CPU idle rate at four representative aggression levels across four scales. Full 10-level sweep available in V11 simulation notebook. Optimal zone 75-90% produces 15-26% improvement over orderly baseline.

Key Finding: The optimal aggression zone of 75-90% is consistent across all tested scales — a universal operating point confirmed by 180 runs per data point. CPU idle drops by 11-26 percentage points, confirming that pressure propagation is active.

Scale	Orderly Baseline	Optimal 85-90%	Improvement	CPU Idle Reduction
100 agents	**151.3 steps**	**128.0**	**15%**	**26pp**
500 agents	**356.7**	**283.9**	**20%**	**16pp**
1000 agents	**518.4**	**397.2**	**23%**	**13pp**
2500 agents	**854.2**	**633.1**	**26%**	**12pp**
5000 agents	**1,273.6**	**940.7**	**26%**	**11pp**

C. Self-Sorting Effect — V13

Convergent random assignment using movement attempt rate as the convergence signal achieves 60% improvement over orderly baseline at 5000 agents — 17-21% better than V12, which used movement success rate. Attempt rate correctly classifies aggressive transactions even when blocked by queue density. Final weight converging to 0.30 is not a convergence failure — it confirms all high-weight transactions completed service, leaving only low-weight transactions that correctly self-identify as passive. Standard deviation of ±9 steps is approximately 4x lower than discrete uniform assignment.

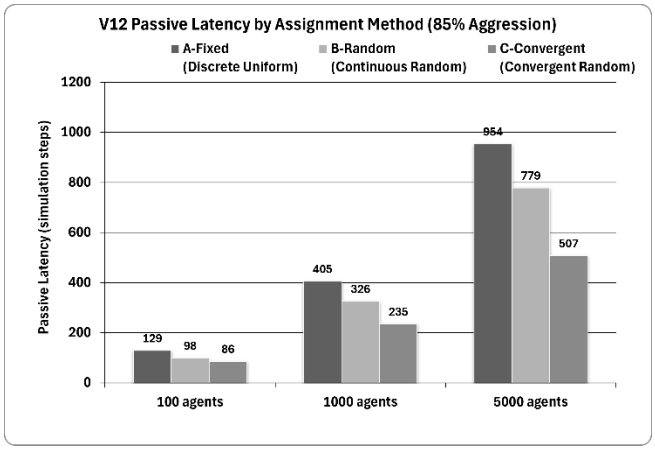

Fig. 3. Passive transaction latency comparison across three assignment methods at 85% aggression. Method C (Convergent Random) achieves 60% improvement over the orderly baseline at 5000 agents with 4x lower variance.

> **Key Finding:** Method C achieves 60% better latency than the orderly baseline at 5000 agents — and is 4x more predictable. The self-sorting effect grows stronger with scale.

D. Complete Performance Summary

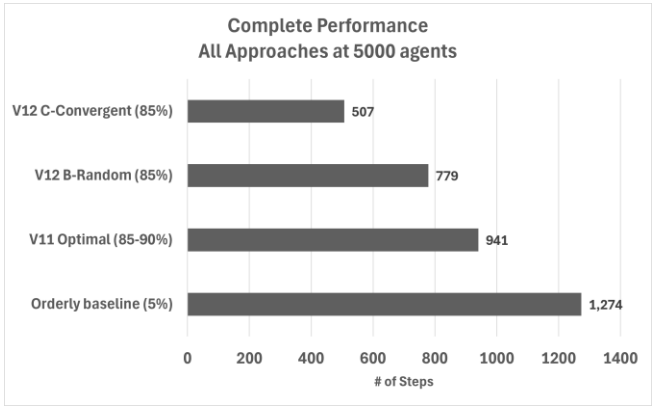

Fig. 4. Complete performance comparison at 5,000 agents. Each bar represents passive transaction latency — shorter is better. Method C achieves 60% reduction over orderly baseline.

> **Key Finding:** From orderly baseline to convergent random admission: 1,274 steps to 507 steps — a 60% reduction in passive transaction latency on identical hardware, achieved through behavioral weight assignment alone.

V. REAL-WORLD VALIDATION — REDIS BENCHMARK

A. Overview

Redis, a widely deployed in-memory data structure server, provides a natural validation environment: multiple concurrent clients compete for a shared resource — the Redis event loop — structurally analogous to the simulated queue. Two parameters map directly to BPQT concepts: client concurrency level (c) — the number of simultaneous Redis clients each sending independent requests, equivalent to the fraction of active agents in the queue; and pipeline depth (P) — the number of commands a client sends without waiting for server acknowledgment, equivalent to the BPQT aggression coefficient. Three experiments were conducted with three independent runs of 200,000 operations each.

An important scope limitation: all results were obtained on a single-node deployment with loopback networking. Absolute throughput numbers reflect this constrained environment. Three behavioral patterns are expected to scale to production cloud environments: the shape of the concurrency-throughput curve, the variance signal preceding thrashing onset, and the dimensionless optimal aggression ratio. Production validation at cloud scale remains an important direction for future work.

B. Concurrency and Pipeline Results

The concurrency sweep (c=1 to c=1000) confirms a stable throughput plateau of approximately 120,000 SET operations per second between c=30 and c=100 clients — the Redis optimal zone. This wide stable operating range confirms the minimum effective pressure principle: sufficient aggression activates pressure propagation without requiring maximum aggression. Above c=200, throughput degrades progressively while P50 latency climbs 38x — thrashing manifests as latency instability before throughput collapse, consistent with simulation findings.

The pipeline depth sweep (P=1 to P=256) confirms the aggression coefficient mapping. Throughput scales from 115,344 req/s at P=1 (coefficient 0.30, passive) to 1,919,313 req/s at P=64 (coefficient 0.96) — a 16x improvement. Above P=64, latency climbs sharply to 4.320ms at P=256, making higher pipeline depths unsuitable for latency-sensitive workloads. The critical finding is the variance signal: at P=8 the coefficient of variation spikes to 67.9% — performance becomes unpredictable before throughput declines. This mirrors the thrashing signal identified in simulation, confirming that instability precedes collapse in both simulation and production systems.

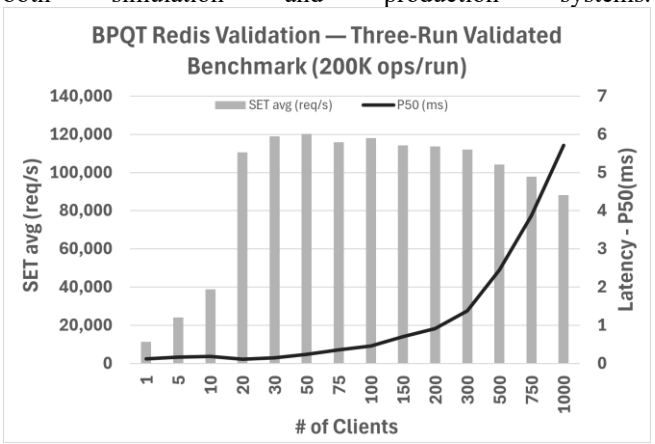

Fig. 5. Redis SET throughput and P50 latency across concurrency levels c=1 to c=1000 (three-run validated, 200,000 ops/run). Throughput plateau at c=30-100 confirms BPQT optimal zone. Latency spike above c=200 confirms thrashing signal.

Key Finding: Redis throughput plateaus at 120,000 req/s between c=30 and c=100 — a wide, stable optimal zone matching simulation prediction. Thrashing manifests as latency instability (38x increase) before throughput collapses.

C. Simulation vs Redis Comparison

Both simulation and Redis benchmark confirm consistent improvement trajectories when normalized to their respective passive baselines:

Scenario	BPQT Equivalent	Simulation	Redis	Direction
Passive baseline	**Orderly 5%**	**1.00x**	**1.00x**	**N/A**
Optimal aggression	**Bunch 15-20%**	**1.35x**	**10.6x**	**YES ✓**
Self-sorting	**C-Convergent**	**2.51x**	**16.7x**	**YES ✓**
Near thrashing	**90%+ boundary**	**Std dev triples**	**CV=67.9%**	**YES ✓**

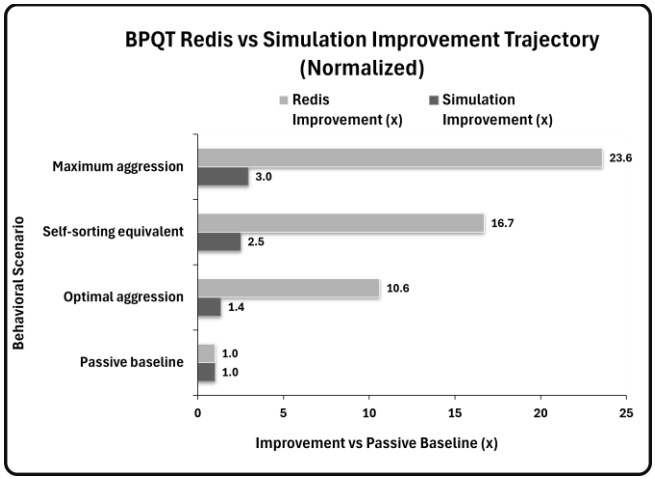

Fig. 6. BPQT simulation vs Redis benchmark results, normalized to passive baseline = 1.0x. Both confirm consistent improvement trajectory across behavioral scenarios.

Key Finding: Both simulation and Redis confirm the same ordering: passive < optimal mix < self-sorting < near thrashing. BPQT behavioral mechanisms operate in production Redis as predicted.

VI. CONCLUSION

Bunch Pressure Queue Theory introduces a two-dimensional behavioral framework for resource scheduling. Starting from two independent observations — young soccer players surrounding a ball before learning positional strategy, and high-density boarding crowds in China — the work formalizes behavioral aggression as an independent scheduling dimension orthogonal to priority, providing the classification layer that current cloud monitoring frameworks lack.

Three simulation campaigns spanning 12,000+ independent runs establish three throughput mechanisms — attentiveness effect, pressure propagation, and the self-sorting effect — with quantitative bounds across five scales. Convergent random assignments using movement attempt rate achieve 60% passive latency improvement with a standard deviation 4x lower than discrete uniform assignment. Redis benchmark validation across 117 files and three independent runs of 200,000 operations confirms that these mechanisms operate in production systems — throughput improves 16x at optimal pipeline depth, and the variance signal predicts thrashing onset before throughput collapses.

This work introduces a concept distinct from existing randomized scheduling literature: randomizing behavioral aggression weights at transaction admission rather than randomizing resource assignments. The resulting self-sorting effect — where transactions with higher initial weights complete service sooner, creating pressure waves for remaining transactions — appears to be a novel contribution. The pre-dispatch readiness concept, where aggressive transaction detection triggers scheduler preparation for subsequent passive transactions, provides a direct implementation pathway from theory to production scheduler design.

> Resource contention is not a problem to be eliminated but a dynamic to be understood, measured, and aligned. Aggressive behavior is beneficial when aligned with priority, pathological when misaligned, and self-organizing when assigned heterogeneously.

A. Limitations

1. Single-node Redis validation only — production cloud validation at scale is the critical next step.
2. Grid-based simulation does not capture network topology effects in distributed systems.
3. Convergent random assignments have a 20-50 step convergence period — not suitable for very short transactions.
4. V13 60% improvement requires independent replication before strong production claims.
5. Whether the self-sorting effect persists across cascaded queues in multi-tier systems is unknown.

B. Future Work

- Production cloud validation — deploy instrumentation on a real cluster and measure predicted vs actual behavior.
- V14 — test at 50,000+ agents when server access is available.
- Multi-node Redis Cluster benchmark — validate behavioral patterns at scale.
- Pre-dispatch scheduler implementation — validate the attentiveness effect as an active optimization.

The author acknowledges the use of Claude (Anthropic) as a collaborative research and writing tool. All conceptual contributions, research direction, empirical observations, simulation design, and intellectual conclusions are the author's own.

Jeffrey Curry received a BSE in Computer Engineering from the University of Michigan (1976) and an MBA in Technology Management from the University of Phoenix (2003). He has more than 50 years of performance measurement experience spanning CAD/CAM workstation analysis at Applicon; biometric AFIS system performance tuning at De La Rue Printrak; TCP/IP stack benchmarking at Network Computing Devices; and storage systems engineering, cache acceleration, and contributions to the Linux iperf network performance tool at Intel Corporation, retiring in 2020. He served as editor and reviewer for the Computer Measurement Group. He currently serves as a Fire Commissioner and Volunteer Firefighter.

---

[1] H. Li, L. Duan, and N. B. Shroff, "When Mobile Crowdsourcing Meets Queueing Systems," IEEE Trans. Netw., 2026.

[2] J. Ghaderi, "Randomized algorithms for scheduling VMs in the cloud," IEEE INFOCOM, 2016.

[3] K. Psychas and J. Ghaderi, "Randomized algorithms for scheduling multi-resource jobs," IEEE/ACM Trans. Netw., vol. 26, no. 5, 2018.

[4] W. Da and E. Kalyvianaki, "Dodoor: Efficient randomized decentralized scheduling," arXiv:2510.12889, 2024.

[5] J. Teahan, "Mesa: Agent-based modeling in Python," v3.5.1, 2024.

[6] A. K. Erlang, "The theory of probabilities and telephone conversations," 1909.

[7] J. D. C. Little, "A proof for the queuing formula: L = lambda W," Operations Research, 1961.

[8] D. Helbing, I. Farkas, and T. Vicsek, "Simulating dynamical features of escape panic," Nature, 2000.

[9] K. Copenhagen et al., "Self-organized sorting limits behavioral variability in swarms," Scientific Reports, 2016.

[10] B. Gregg, "Systems Performance: Enterprise and the Cloud," 2nd ed., Pearson, 2020.

[11] S. Sanfilippo, "Redis: In-memory data structure store," Redis Labs, 2024.

[12] A. Sinko, S. Nikolova, and M. Sutton, "Targets for maximum waiting times and patient prioritization," 2013.